
Junior Cloud Security Engineer

Internship Task Report

Intern Full Name	Sukhmandeep Kaur
Intern Email	[REDACTED]
Cohort	Cohort 7
Task ID	CSE-AWS-L-014-04
Task Title	AWS Secrets Manager Implementation and Access Control Enforcement
Skill Domain	AWS Cloud Security / Secrets Management
Difficulty Level	Intermediate
CPE Credits	04 CPE Credits
Guiding Framework(s)	MITRE ATT&CK / IAM least privilege / AWS security best practices
Submission Date	30 August 2026

1.0 Task Overview

Task ID	CSE-AWS-L-014-04
Task Title	AWS Secrets Manager Implementation and Access Control Enforcement
Skill Domain	AWS Cloud Security / Secrets Management
Difficulty	Intermediate
Duration	12 Hours
CPE Credits	04 CPE Credits
Cloud Platform(s)	AWS Free Tier account; Cybr authorised hosted labs
AWS Region(s) Used	us-east-2
Primary Tools	AWS Console, AWS Secrets Manager, AWS IAM, AWS Lambda, ChatGPT for explanation/documentation support
Target Environment	Personal AWS lab account and authorised Cybr training labs
MITRE ATT&CK IDs	T1552.001 (Credentials In Files); T1078 (Valid Accounts); T1484.002 (Domain Trust Modification - task brief mapping)
Guiding Frameworks	MITRE ATT&CK; AWS IAM least-privilege principles; AWS Secrets Manager security practices
Regulatory Alignment	Access control, credential lifecycle management, encryption, logging, and secure secret handling

1.1 Introduction

This task focused on securely storing, controlling, retrieving, and rotating application credentials with AWS Secrets Manager. The lab addressed common credential risks such as hard-coded secrets, excessive IAM permissions, long-lived credentials, and unauthorised secret retrieval, while demonstrating how KMS encryption, least privilege, Lambda integration, rotation, and logging reduce exposure.

1.2 Objectives & Learning Outcomes

The task was completed through the following measurable outcomes:

- Understand Secrets Manager encryption, secret lifecycle, managed secrets, and ARN-scoped IAM access - completed through the Phase A conceptual review and summary.
- Create AppDBCredentials and enforce least-privilege access - completed by creating the secret and LambdaSecretAccessRole with GetSecretValue limited to the secret ARN.
- Retrieve the secret securely from Lambda - completed by configuring RetrieveSecretFunction to return only the username and not the plaintext password.

- Assess rotation and exposure risks - completed through 30-day rotation configuration work, troubleshooting, threat modelling, and a likelihood-impact risk matrix.
- Reinforce skills through hosted labs and professional reporting - completed through the Cybr lab activities and consolidation of evidence and findings in this report.

1.3 Scope & Legal Authorisation

All technical activities were performed only in authorised training environments for this internship task.

Authorised environments used:

- Personal AWS lab account in us-east-2 using Secrets Manager, IAM, and Lambda.
- Cybr: Introduction to Secrets Manager Enumeration - authorised hands-on lab environment.
- Cybr: Access Secrets Manager via Lambda Function with LFI - authorised hands-on lab environment.
- No production or customer environment was used.

No external AWS accounts, production systems, third-party cloud resources, or real customer data were intentionally accessed. AI assistance was used for explanation, troubleshooting guidance, policy review, and report drafting; AWS console actions and screenshots were performed by the intern.

1.4 MITRE ATT&CK Cloud Threat Mapping

MITRE ATT&CK Technique ID	Technique Name	Relevance to This Task
T1552.001	Credentials In Files	Relevant to preventing credentials from being embedded in application files or configuration; Secrets Manager provides managed storage and retrieval.
T1078	Valid Accounts	Stolen or exposed valid credentials can be abused for authenticated access; rotation and least privilege reduce their useful lifetime and blast radius.
T1484.002	Domain Trust Modification	Included in the task metadata. The hands-on lab did not modify domain trust; its direct applicability to this Secrets Manager implementation was limited.

2.0 Environment & Tool Setup

2.1 Cloud Architecture Overview

The lab used an AWS account in us-east-2. Secrets Manager stored AppDBCredentials, IAM provided an execution role restricted to the secret ARN, and Lambda retrieved the secret at runtime. Phase D extended the design with rotation and threat modelling, while Cybr labs provided authorised practice with secret enumeration and Lambda-related access scenarios.

Cloud Resource / Component	Type / Service	Role in Task	Region / Config Notes
AppDBCredentials	AWS Secrets Manager	Stores application database username/password securely	us-east-2; encrypted with aws/secretsmanager
aws/secretsmanager	AWS KMS managed key	Encrypts the secret at rest	Default Secrets Manager KMS key
LambdaSecretAccessRole	AWS IAM role	Lambda execution role with GetSecretValue restricted to one secret ARN	Trusted service: Lambda
RetrieveSecretFunction	AWS Lambda	Retrieves AppDBCredentials at runtime and returns username only	Python runtime; us-east-2
Rotation Lambda / configuration	Secrets Manager + Lambda	Supports scheduled secret rotation and exposure reduction	30-day rotation objective; invocation permission troubleshooting

2.2 AWS CLI & Tool Configuration

The task was primarily console-driven. The following services and tools were used during the work documented in this report.

Tool / Service	Version	Installation Method	Purpose in This Task
AWS Console	Web console	Browser-based; no installation required	Create and configure Secrets Manager, IAM, and Lambda resources
AWS Secrets Manager	AWS managed	AWS Console	Store, retrieve, and configure rotation for AppDBCredentials
AWS IAM	AWS managed	AWS Console	Create LambdaSecretAccessRole and ARN-scoped inline policy
AWS Lambda	Python runtime	AWS Console	Run RetrieveSecretFunction and rotation-related Lambda configuration
Cybr	Hosted lab platform	Browser-based	Authorised Secrets Manager enumeration and Lambda/LFI practice
ChatGPT	GPT-5.6 Sol	Web application	Explain concepts, troubleshoot errors, review policy syntax, and assist report drafting

Key Configuration Notes

The implementation was completed through the AWS Console. The key configuration values were: region us-east-2; secret AppDBCredentials; execution role LambdaSecretAccessRole; function RetrieveSecretFunction; and an IAM permission limited to secretsmanager:GetSecretValue on the AppDBCredentials ARN.

2.3 AI Tool Usage Declaration

ChatGPT was used to explain AWS Secrets Manager concepts, troubleshoot Lambda runtime and rotation-permission errors, review IAM policy structure, and assist with professional documentation. It was not used to operate the AWS Console, create screenshots, or perform hosted-lab actions.

AI Tool	Used For	What Was NOT Done with AI
ChatGPT	Concept explanation, troubleshooting guidance, policy review, threat-model/report drafting	Did not operate AWS resources, complete hosted labs, or generate AWS console screenshots.

All technical findings, CLI outputs, configurations, and evidence in this report are the intern's own work.

3.0 Methodology — Phase Execution

Phase A: Theory & Conceptual Foundation

Phase Objective	Understand the purpose of AWS Secrets Manager and its role in secure credential management.
Estimated Time	60 minutes
Evidence File(s)	PhaseA_Overview.pdf

Actions Taken

Reviewed how Secrets Manager encrypts secret values at rest through AWS KMS, compared managed secrets with environment variables, studied the creation-rotation-retrieval-deletion lifecycle, and examined how IAM resource ARNs can restrict secret access. The conceptual output was consolidated into a one-page overview.

Key Observations

Managed secrets provide stronger centralised control than embedding credentials in code or environment configuration. KMS protects stored values, IAM determines who can retrieve them, rotation limits credential lifetime, and deletion controls support lifecycle governance.

Evidence Produced

- PhaseA_Overview.pdf - one-page summary of KMS encryption, managed secrets, lifecycle, and IAM ARN-based access control.

Phase B: Cloud Environment Setup & Target Preparation

Phase Objective	Prepare your AWS environment for secure secret management.
Estimated Time	90 minutes
Evidence File(s)	PhaseB_Setup.png; IAMPolicy.json

Actions Taken

Created the AppDBCredentials secret in AWS Secrets Manager with username and password key-value pairs and selected the default aws/secretsmanager KMS key. Recorded the secret ARN. Created LambdaSecretAccessRole with Lambda as the trusted service and attached an inline policy allowing only secretsmanager:GetSecretValue against the AppDBCredentials ARN.



Key Observations

The setup demonstrated resource-scoped least privilege: the Lambda role did not require wildcard access to all secrets. The default KMS integration simplified encryption at rest while the secret ARN provided a precise resource boundary for IAM.

Evidence Produced

- PhaseB_Setup.png - Secrets Manager evidence showing AppDBCredentials and its ARN.
- PhaseB_IAMPolicy.json - inline policy granting GetSecretValue only on the AppDBCredentials secret.

Phase C: Core Execution

Phase Objective	Retrieve stored secrets securely from a Lambda function using the configured role.
-----------------	--

Estimated Time	120 minutes
Evidence File(s)	PhaseC_LambdaTest.png

Actions Taken

Created RetrieveSecretFunction and assigned LambdaSecretAccessRole. An initial test failed because the function was created with a Node.js runtime while Python boto3 code had been inserted. The runtime was corrected to Python, the handler was set appropriately, the code was deployed, and the function was tested to retrieve AppDBCredentials. The response was designed to return only the username so the plaintext password was not exposed in the test output or logs.

Key Observations

The runtime mismatch produced a clear Runtime.UserCodeSyntaxError, showing the importance of matching code language to Lambda runtime configuration. After correction, the design demonstrated secure runtime retrieval through an IAM role rather than hard-coded application credentials.

Evidence Produced

- PhaseC_LambdaTest.png - successful Lambda test output showing username retrieval without exposing the password.

Phase D: Independent Extension & Advanced Implementation

Phase Objective	Implement automatic rotation and develop a threat model for potential exposure scenarios.
Estimated Time	150 minutes
Evidence File(s)	PhaseD_ThreatModel.pdf; RotationConfig.png

Actions Taken

Configured the secret for a 30-day rotation schedule and worked through the rotation Lambda integration. During testing, Secrets Manager reported that it could not invoke the specified Lambda function because the function resource policy did not grant secretsmanager.amazonaws.com permission to invoke it. The required resource-based permission was identified. A threat model and risk matrix were produced covering database credentials, unauthorised access, IAM misconfiguration, long-lived credential exposure, encryption, rotation, and logging.

Key Observations

Rotation reduces the useful lifetime of exposed credentials, but rotation is not only a schedule: the rotation function must have correct resource-based invocation permissions and must be able to update the backing credential safely. The exercise also showed that IAM misconfiguration remains a high-impact risk even when encryption is enabled.

Evidence Produced

- PhaseD_ThreatModel.pdf - threat model and likelihood-impact risk matrix.
- RotationConfig.png - evidence of the configured rotation schedule/function.

Risk Matrix (Phase D Output)

Cloud Asset / Resource	Threat / Risk	MITRE Ref.	Likelihood (1–5)	Impact (1–5)	Score	Rating
AppDBCredentials	Unauthorised secret retrieval	T1078	3	5	15	Critical
LambdaSecretAccessRole	Misconfigured or overly broad IAM access	T1078	3	5	15	Critical
Application credentials	Credential exposure in code/logs	T1552.001	3	5	15	Critical
Rotation configuration	Long-lived credential if rotation is disabled/fails	T1078	3	4	12	High

Score = Likelihood × Impact | 1–4: Low | 5–9: Medium | 10–14: High | 15–25: Critical

Phase E: Hosted Lab Integration

Phase Objective	Use the following external labs to reinforce learning.
Estimated Time	180 minutes
Evidence File(s)	LabCompletion.png

Actions Taken

Completed the Cybr hosted-lab activities focused on Secrets Manager enumeration and accessing Secrets Manager through a Lambda-related scenario with LFI. The exercises were performed only within the authorised Cybr environment and reinforced how exposed application paths or excessive permissions can lead to secret discovery or retrieval.

Key Observations

The hosted labs connected defensive configuration to attacker behaviour: secret names, permissions, and application integration paths can become useful discovery points when IAM or application controls are weak. Least privilege and avoiding unnecessary secret exposure to application components reduce the available attack path.

Evidence Produced

- LabCompletion.png - screenshot(s) of Cybr lab completion badges for the required hosted labs.

Phase F: Documentation & Reporting

Phase Objective	Compile all findings into a professional report suitable for cloud security review boards.
Estimated Time	Documentation phase
Evidence File(s)	CSE_CSE-AWS-L-014-04_SK_20260829.pdf

Actions Taken

Consolidated Phase A-F activities, evidence references, findings, threat model, risk scores, remediation actions, and reflection answers into the EncryptEdge Labs internship report format. Sensitive credential values were excluded from the report and evidence references were kept separate from secret contents.

Key Observations

The final review showed that secure secret management depends on multiple controls working together: managed storage, encryption, precise IAM permissions, secure application retrieval, rotation, and monitoring. A failure in any one layer can increase the blast radius of credential compromise.

Evidence Produced

- CSE_CSE-AWS-L-014-04_SK_20260829.pdf - final internship task report.

4.0 Findings

This section records the significant security outcomes and configuration issues observed during the task, including both weaknesses requiring remediation and controls that were successfully implemented.

4.1 Findings Summary

#	Finding Title	Type / Category	Cloud Service / Resource	MITRE Ref.	Phase	Severity	Evidence Ref.
1	Least-privilege secret access enforced	Access Control / Positive Control	IAM role + Secrets Manager	T1078	B/C	Info	IAMPolicy.json; PhaseC_LambdaTest.png
2	Lambda runtime mismatch blocked execution	Configuration Error	RetrieveSecretFunction	N/A	C	Low	Phase C test/troubleshooting record
3	Rotation Lambda invocation permission missing	Access Control Misconfiguration	Secrets Manager rotation Lambda	T1078	D	High	Rotation configuration/error observation
4	Credential exposure risk if secrets are embedded or long-lived	Credential Management Risk	Application credentials	T1552.001	A/D/E	High	PhaseA_Overview.pdf; PhaseD_ThreatModel.pdf

4.2 Detailed Finding Analysis

Three detailed findings are documented below, prioritising the rotation permission gap, credential exposure risk, and the successful least-privilege implementation.

Finding 1: Rotation Lambda Invocation Permission Missing

Finding Title	Rotation Lambda invocation permission missing
Type / Category	Access Control Misconfiguration
Cloud Service / Resource	AWS Secrets Manager rotation Lambda
Description	Secrets Manager could not invoke the selected rotation Lambda because the function resource policy did not allow the secretsmanager.amazonaws.com service principal to invoke it.
How Identified	Observed while enabling/testing automatic rotation in the Secrets Manager console.
MITRE ATT&CK Reference	T1078 (Valid Accounts) - related credential-control context; primary issue is an AWS resource-policy configuration gap.

Business / Security Impact	Rotation cannot complete, leaving credentials unchanged for longer than intended and increasing exposure duration if a credential is compromised.
Severity / Impact	High
Evidence Reference	RotationConfig.png / Phase D troubleshooting observation

Finding 2: Credential Exposure Risk from Unmanaged or Long-Lived Secrets

Finding Title	Credential exposure risk from unmanaged or long-lived secrets
Type / Category	Credential Management Risk
Cloud Service / Resource	AppDBCredentials / application configuration
Description	Credentials stored in files, code, logs, or unmanaged environment configuration can be exposed and remain usable for an extended period. Secrets Manager, secure retrieval, and rotation reduce this risk.
How Identified	Identified through Phase A analysis, Phase D threat modelling, and hosted lab scenarios.
MITRE ATT&CK Reference	T1552.001 (Credentials In Files)
Business / Security Impact	Exposed credentials can allow unauthorised application or database access, potentially affecting confidentiality and integrity.
Severity / Impact	High
Evidence Reference	PhaseA_Overview.pdf; PhaseD_ThreatModel.pdf; LabCompletion.png

Finding 3: Least-Privilege Secret Retrieval Successfully Implemented

Finding Title	Least-privilege secret retrieval successfully implemented
Type / Category	Access Control / Positive Control
Cloud Service / Resource	LambdaSecretAccessRole and AppDBCredentials
Description	The Lambda execution role was restricted to secretsmanager:GetSecretValue for the specific AppDBCredentials ARN rather than all secrets. The Lambda design returned only the username.
How Identified	Validated through the IAM inline policy and Lambda retrieval test.
MITRE ATT&CK Reference	T1078 (Valid Accounts) - least privilege reduces abuse potential of compromised valid identities.

Business / Security Impact	Restricting the role reduces blast radius because compromise of the Lambda role does not automatically grant access to unrelated secrets.
Severity / Impact	Informational
Evidence Reference	IAMPolicy.json; PhaseC_LambdaTest.png

5.0 Risk Assessment & Cloud Threat Model

The Phase D threat model was used to connect observed configuration issues with defensive coverage and to assess how credential risk changes over time.

5.1 Cloud Security Control Gaps & Defensive Coverage

Finding / Gap	Observable / Trigger in Cloud Telemetry	Recommended Control	AWS Service / Tool for Implementation
Rotation Lambda invocation permission missing	Failed rotation attempts and Lambda invocation errors; CloudTrail can record related API activity	Grant scoped <code>lambda:InvokeFunction</code> permission to <code>secretsmanager.amazonaws.com</code> and validate rotation end-to-end	Lambda resource-based policy; Secrets Manager; CloudTrail
Unmanaged/long-lived credential exposure	GetSecretValue activity, source identity/IP, unusual retrieval frequency	Use Secrets Manager, enable rotation, alert on unusual GetSecretValue calls, and prevent secrets in code	Secrets Manager; CloudTrail; CloudWatch
IAM role misconfiguration risk	IAM policy changes and unexpected secret-access attempts	Maintain ARN-scoped policies, review role permissions, and use Access Analyzer/policy review	IAM; IAM Access Analyzer; CloudTrail

5.2 Risk Exposure Over Time

The highest risks are credential exposure, overly broad access, and failed/disabled rotation. Their effect increases over time because compromised credentials remain useful until revoked or rotated and excessive permissions expand what a compromised workload can access.

Risk 1 - Long-lived or exposed credentials:

If AppDBCredentials were exposed and rotation remained disabled or non-functional, an attacker could continue using the same credential until it was manually changed. The Phase D rotation issue therefore directly increases persistence risk and makes rapid remediation more important.

Risk 2 - Misconfigured IAM permissions:

If `LambdaSecretAccessRole` were broadened from the specific secret ARN to wildcard secret access, compromise of the function or role could expose additional secrets. The implemented ARN restriction reduces this blast radius and should be preserved as the environment scales.

Risk 3 - Insufficient monitoring of secret retrieval:

Without reviewing CloudTrail activity or alerting on unusual `GetSecretValue` calls, unauthorised retrieval could remain unnoticed. Detection should focus on unexpected principals, source locations, access patterns, and policy changes affecting secret access.

6.0 Recommendations

Recommendations below trace directly to the findings and risks identified in this task.

6.1 Technical Remediation

#	Finding	Specific Remediation Action (with Policy / CLI where applicable)	Priority	Standard / Reference
1	Rotation Lambda invocation permission missing	Add a resource-based Lambda permission allowing <code>secretsmanager.amazonaws.com</code> to invoke only the approved rotation function, scoped to the <code>AppDBCredentials</code> source ARN where supported; then retest rotation.	Immediate	AWS Secrets Manager rotation security / least privilege
2	Credential exposure risk	Keep credentials out of source code and logs; retrieve them at runtime from Secrets Manager and rotate them on a defined schedule.	Immediate	MITRE ATT&CK T1552.001; AWS Secrets Manager practices
3	IAM misconfiguration risk	Retain <code>secretsmanager:GetSecretValue</code> only on the <code>AppDBCredentials</code> ARN and review changes that introduce <code>Resource: "*" or unrelated secret ARNs.</code>	Short-term	Least privilege; MITRE ATT&CK T1078
4	Monitoring gap	Use CloudTrail visibility and CloudWatch alerting for unusual <code>GetSecretValue</code> , <code>PutResourcePolicy</code> , <code>UpdateSecret</code> , <code>RotateSecret</code> , and IAM policy-change activity.	Short-term	AWS logging and monitoring practices

6.2 Strategic & Operational Improvements

Beyond the individual fixes, the organisation should standardise secret naming and tagging by environment, use separate IAM roles for applications and environments, automate rotation validation, centralise CloudTrail monitoring for Secrets Manager API calls, and review secret-access policies regularly for wildcard permissions or unnecessary principals.

- Establish a standard pattern for application secrets: Secrets Manager storage, KMS encryption, ARN-scoped IAM access, runtime retrieval, and rotation testing.
- Separate development, test, and production secrets and roles so one compromised workload cannot retrieve secrets from another environment.
- Alert on unusual `GetSecretValue` activity and on changes to secret resource policies, rotation configuration, and IAM permissions.

- Periodically test rotation functions and verify that both Secrets Manager invocation permissions and backing-service credential updates succeed.
- Include secret scanning and policy review in CI/CD to reduce the chance of credentials being committed to source code or overly broad IAM policies reaching production.

7.0 Reflection

Q1 How does enforcing least privilege reduce the blast radius of compromised credentials?

Least privilege limits what a compromised identity can access. In this lab, `LambdaSecretAccessRole` was allowed to call `GetSecretValue` only for `AppDBCredentials` rather than every secret in the account. If that Lambda role were compromised, the attacker would have a much smaller set of accessible resources. This containment reduces the potential confidentiality and business impact of a single credential or workload compromise.

Q2 What are potential business impacts if secret rotation is disabled?

If rotation is disabled, an exposed credential can remain valid for a long period, increasing the opportunity for unauthorised access. This can lead to data exposure, service misuse, incident-response costs, audit findings, and loss of customer trust. The Phase D exercise also showed that a configured schedule is not enough if the rotation Lambda cannot be invoked successfully.

Q3 How would you detect unauthorized attempts to retrieve secrets from your account?

I would monitor AWS CloudTrail for Secrets Manager API activity such as `GetSecretValue` and correlate events with the calling principal, source IP, time, and expected application behaviour. CloudWatch metric filters or other monitoring rules can alert on unusual retrieval patterns, repeated access failures, or access from unexpected identities. IAM and secret-policy changes should also be monitored because attackers may first try to expand permissions.

Q4 If an application requires multiple secrets across environments, how would you design scalable access control?

I would separate secrets by environment and application using consistent names and tags, and give each workload its own IAM role. Policies would grant only the required actions against the specific secret ARNs or carefully controlled tag-based resource sets for that workload. Development roles would not receive production-secret access. This keeps permissions manageable while preserving isolation and least privilege as the number of secrets grows.

Q5 How would you communicate a discovered misconfiguration in production to management?

I would explain the issue in business terms: what resource is affected, what could happen, how likely and severe the impact is, and whether there is evidence of exploitation. I would avoid unnecessary technical detail in the executive message but provide a clear remediation owner, priority, and timeline. For a rotation failure, I would state that credentials may remain valid longer than policy intends and recommend restoring rotation, reviewing access logs, and rotating the affected credential safely.

8.0 Deliverables Submission Checklist

The following deliverables correspond to the evidence requirements supplied for Phases A-F.

✓	Phase	Deliverable Description	Your Filename	Status
✓	Phase A	Theory overview	PhaseA_Overview.pdf	Complete
✓	Phase B	Secrets Manager setup and IAM policy	PhaseB_Setup.png; PhaseB_IAMPolicy.json	Complete
✓	Phase C	Secure Lambda retrieval test	PhaseC_LambdaTest.png	Complete
✓	Phase D	Rotation configuration and threat model	RotationConfig.png; PhaseD_ThreatModel.pdf	Complete
✓	Phase E	Cybr hosted-lab completion evidence	LabCompletion.png	Pending
✓	Report	Final internship task report	CSE_CSE-AWS-L-014-04_SK_20260829.pdf	Complete

9.0 Additional Observations

Two troubleshooting events were significant during the task:

- Lambda initially used a Node.js runtime with Python code, producing `Runtime.UserCodeSyntaxError: Unexpected token import`. The runtime/handler configuration was corrected to Python before retesting.
- Secrets Manager rotation initially failed because the selected rotation Lambda did not grant the `secretsmanager.amazonaws.com` principal permission to invoke the function. The required resource-based Lambda permission was identified as the remediation.
- A Redshift Serverless namespace subscription warning appeared in the AWS console during secret setup; it was unrelated to the Secrets Manager objective and did not change the secret-management design.
- The hosted labs reinforced how application-level weaknesses and permission scope can combine with secret discovery to create a credential exposure path.

Declaration & Sign-Off

I confirm that:

- All work in this report is my own except where AI usage is explicitly declared in Section 2.3.
- All technical activities were performed exclusively within the authorised cloud environments listed in Section 1.3.
- No external AWS accounts, production environments, third-party cloud resources, or real customer data were accessed during this task.
- All AWS credentials, access keys, session tokens, and account identifiers used during this task have been redacted in evidence files before submission.
- I have reviewed this report for accuracy, completeness, and professional presentation before submission.

Intern Full Name	Signature	Date of Submission
Sukhmandeep Kaur	__Sukhmandeep Kaur_____	30 August 2026

EncryptEdge Labs Limited — Junior Cloud Security Engineer Internship Programme

Company No. 15830711 | United Kingdom | encryptedgelabs.com

Copyright © 2026 EncryptEdge Labs. All rights reserved. | Report Template v3.0